

/*****

* Final Project: Hotel Reservation System - Project Documentation

* Course: APD545 NAA - Winter 2026

* This assignment represents our own work in accordance with Seneca Academic Policy.

* Name: Dohyun Lim, Phuong Bac Nguyen, Eunki Kim

* Group: #18

* Date: Apr 6, 2026

*****/

NewnhamNexus Hotel Reservation System

Project Documentation

Table of Contents

<i>NewnhamNexus Hotel Reservation System.....</i>	<i>1</i>
Project Documentation.....	1
1. Project Overview	2
1.1 Summary.....	2
1.2 Key Features	3
1.3 Technologies Used	3
2. Architecture Summary	4
2.1 Three-Tier Architecture	4
2.2 Cross-Cutting Concerns	4
2.3 MVC Pattern	4
2.4 Dependency Injection	4
2.5 ORM Implementation	5
3. Design Artifacts	6
3.1 Class Diagram	6
3.2 Sequence Diagram - Kiosk Booking Flow	8
3.3 Deployment Diagram	9
3.4 Package Diagram	9
3.5 UI Screenshots	11
4. Entity and Relationship Mapping	22
4.1 Entities (9 Tables + 2 Join Tables)	22
4.2 Relationships.....	22
4.3 Key JPA Annotations Used	23
4.4 Cascade and Fetch Strategy Notes	24

5. Pattern Usage	24
5.1 Singleton Pattern	24
5.2 Factory Pattern	25
5.3 Strategy Pattern	25
5.4 Observer Pattern	26
5.5 Decorator Pattern	27
6. Business Rules	28
6.1 Room Occupancy Rules	28
6.2 Pricing Rules	28
6.3 Discount Rules	28
6.4 Loyalty Program Rules (Nexus Rewards)	29
6.5 Feedback Rules	29
6.6 Reservation Status Flow	29
7. Security and Logging	29
7.1 Authentication	29
7.2 Role-Based Access Control	29
7.3 Logging Configuration	30
7.4 Exception Handling	30
8. Export and Reporting	31
8.1 Report Types	31
8.2 Export Formats	31
8.3 Sample Report Structure	31
8.4 PDF Export Implementation	32
9. Challenges and Learnings	32
9.1 Team Challenges	32
9.2 Individual Reflections	33
9.3 Suggestions for Future Improvements	34
Appendices	34
A. File Structure	34
B. How to Run	35
C. References	35

1. Project Overview

1.1 Summary

NewnhamNexus Hotel is a famous tourist hotel in North York, Toronto. This project replaces the existing manual reservation system with a computerized solution to address the following problems:

- Records vulnerable to destruction by fire/disaster or theft
- Storage requiring extra cabinet space
- Difficult and time-consuming record searches
- Error-prone manual billing system

The system includes **2 self-service kiosks** for contactless booking, an **admin management portal**, and a **guest feedback system**.

1.2 Key Features

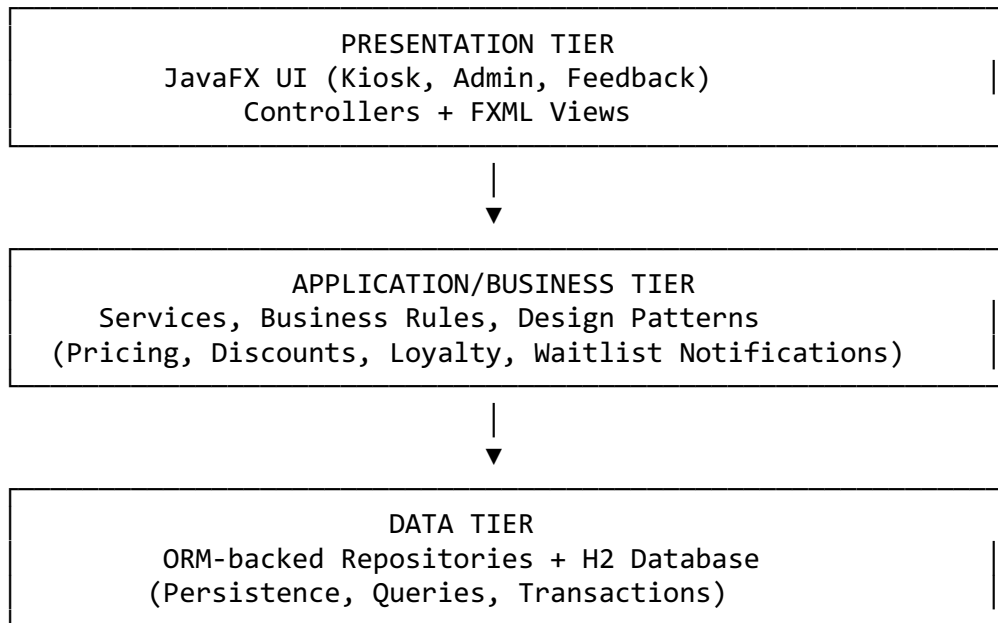
Module	Features
Kiosk	6-step booking flow, room selection, add-on services, loyalty enrollment
Admin Portal	CRUD operations, payments, checkout, waitlist management, reporting
Feedback	1-5 star ratings, comments, sentiment tagging
Loyalty Program	Nexus Rewards - earn/redeem points
Reporting	Revenue, occupancy, feedback reports with CSV/PDF export

1.3 Technologies Used

Technology	Version	Purpose
Java	17+	Core programming language
JavaFX	21	Desktop UI framework
Hibernate	6.4.4	JPA/ORM implementation
H2 Database	2.2.224	Embedded file-based database
BCrypt (jbcrypt)	0.4	Password hashing
Apache PDFBox	2.0.31	PDF report generation
Maven	3.6+	Build and dependency management

2. Architecture Summary

2.1 Three-Tier Architecture



2.2 Cross-Cutting Concerns

Concern	Implementation
Logging	Rotating file logs (1MB limit, 10 files max) in /logs directory
Security	BCrypt password hashing, role-based access control
Configuration	persistence.xml for JPA, styles.css for UI theming
Exception Handling	Try-catch blocks with user-friendly error alerts

2.3 MVC Pattern

Component	Implementation
Model	JPA Entity classes (Guest, Room, Reservation, etc.)
View	FXML files in /resources/view/
Controller	KioskController, AdminController, FeedbackController

2.4 Dependency Injection

The ApplicationContext class serves as a simple dependency injection container:

```

public class AppContext {
    private static final RoomRepository roomRepository = new RoomRepository();
    private static final GuestRepository guestRepository = new GuestRepository();
    private static final ReservationRepository reservationRepository = new ReservationRepository();
    // ... other repositories

    private static final ReservationService reservationService = new ReservationService(
        reservationRepository, roomRepository, guestRepository, addOnRepository);
    private static final PricingService pricingService = new PricingService();
    // ... other services

    // Getters for each dependency
    public static ReservationService getReservationService() { return reservationService; }
}

```

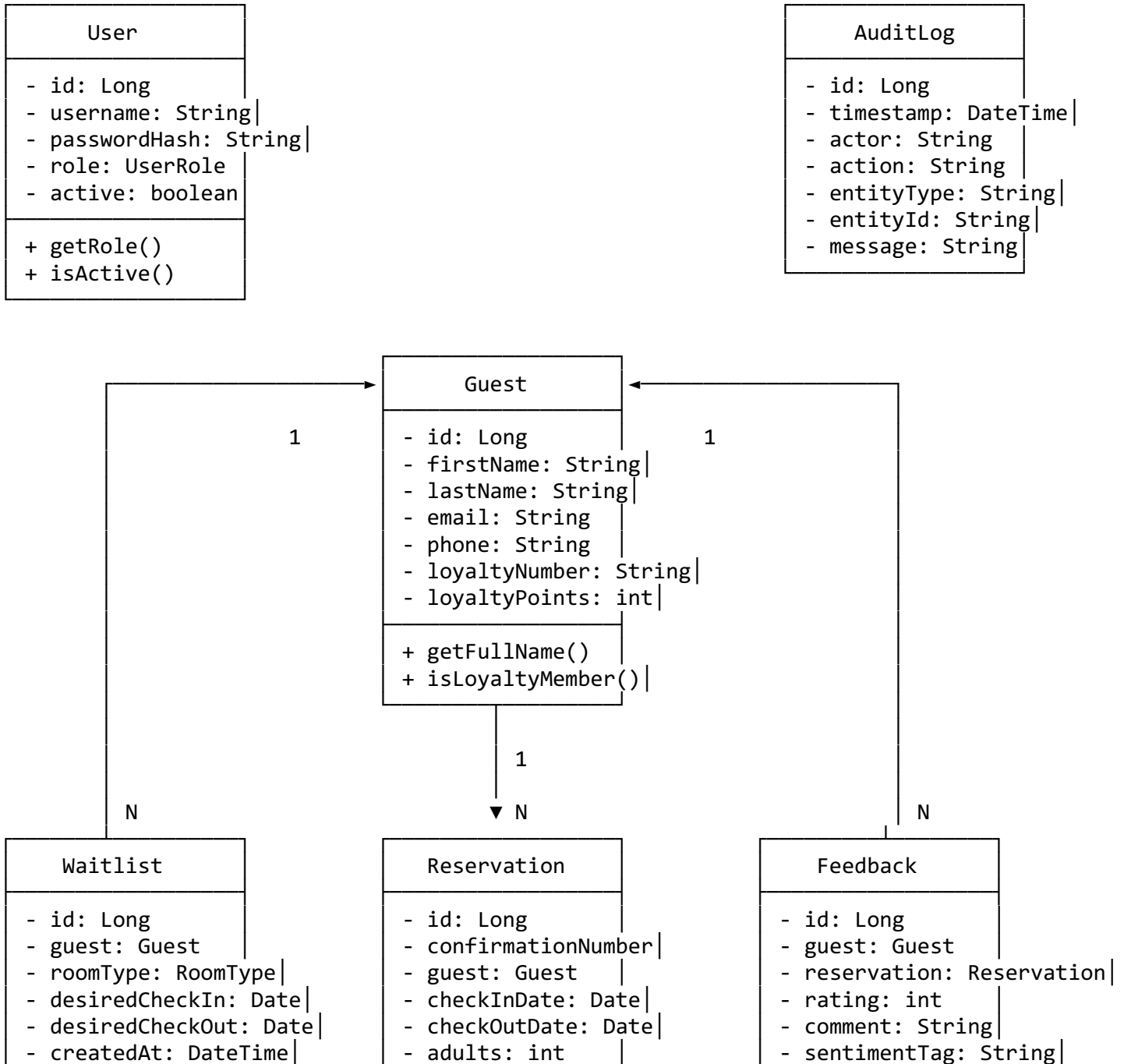
2.5 ORM Implementation

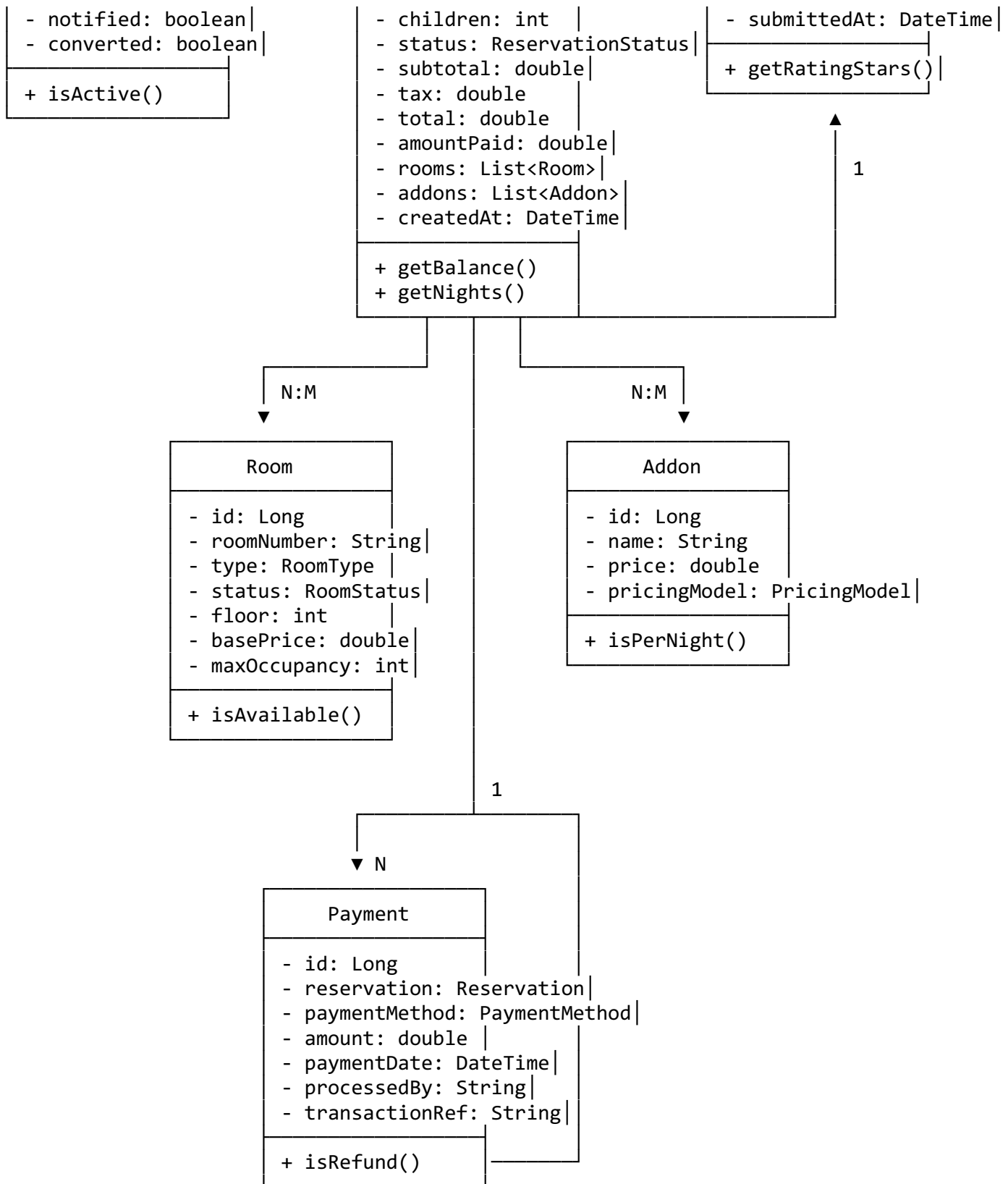
- **Framework:** Hibernate 6.4.4 with Jakarta Persistence API
- **Database:** H2 embedded (file: ./data/hoteldb.mv.db)
- **Schema Generation:** hibernate.hbm2ddl.auto = update
- **Connection Pooling:** Hibernate default connection provider

3. Design Artifacts

3.1 Class Diagram

ENTITY CLASS DIAGRAM



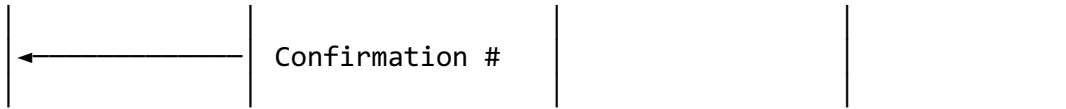


RELATIONSHIP SUMMARY

Guest (1) —>< (N) Reservation	: One guest can have many reservations
Guest (1) —>< (N) Feedback	: One guest can leave many feedbacks
Guest (1) —>< (N) Waitlist	: One guest can have many waitlist entries
Reservation (1) —>< (N) Payment	: One reservation can have many payments
Reservation (1) —>< (1) Feedback	: One reservation has one feedback
Reservation (N) >—< (M) Room	: Many-to-many via reservation_rooms
Reservation (N) >—< (M) Addon	: Many-to-many via reservation_addons

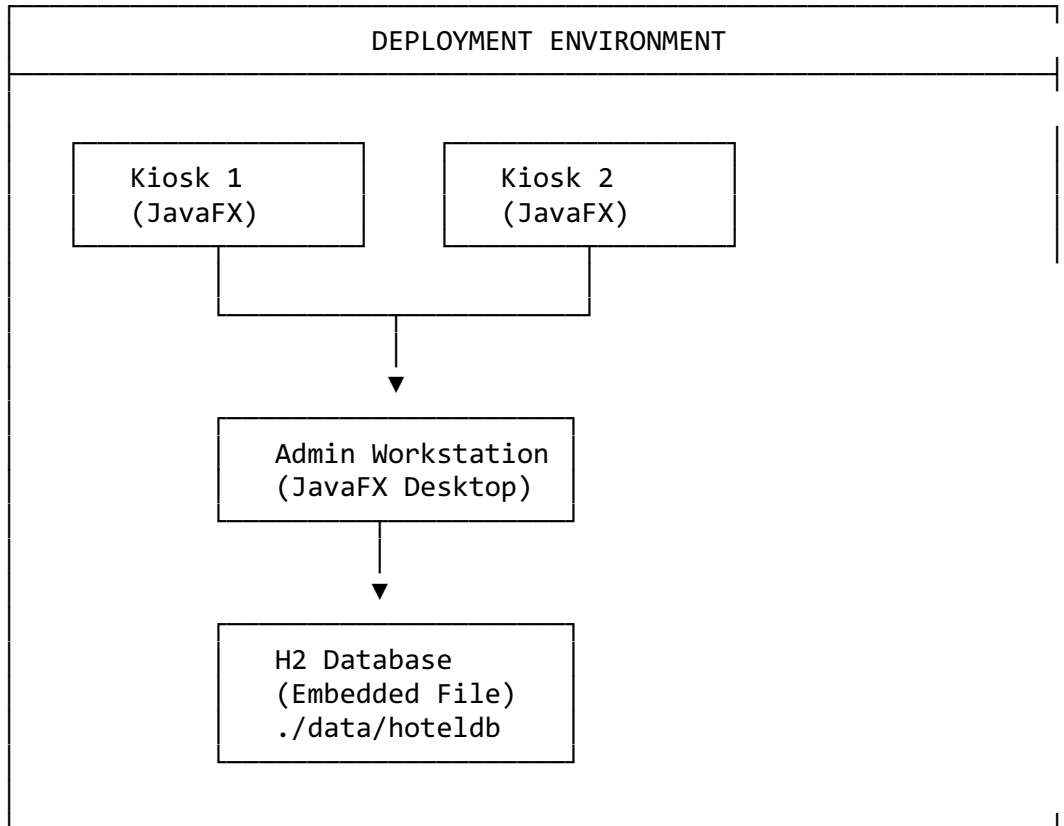
3.2 Sequence Diagram - Kiosk Booking Flow





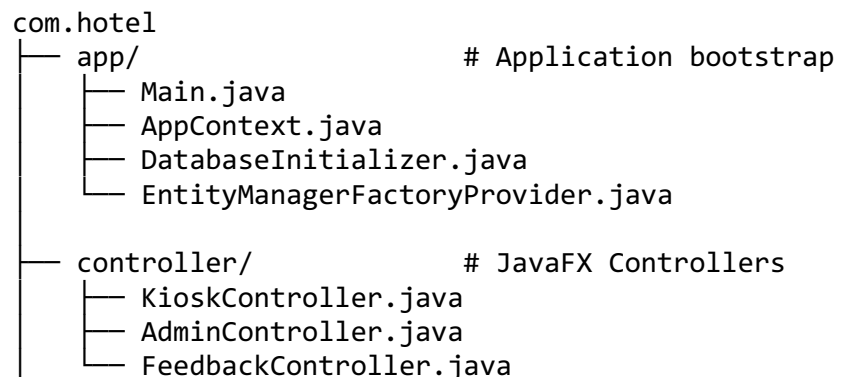
3.3 Deployment Diagram

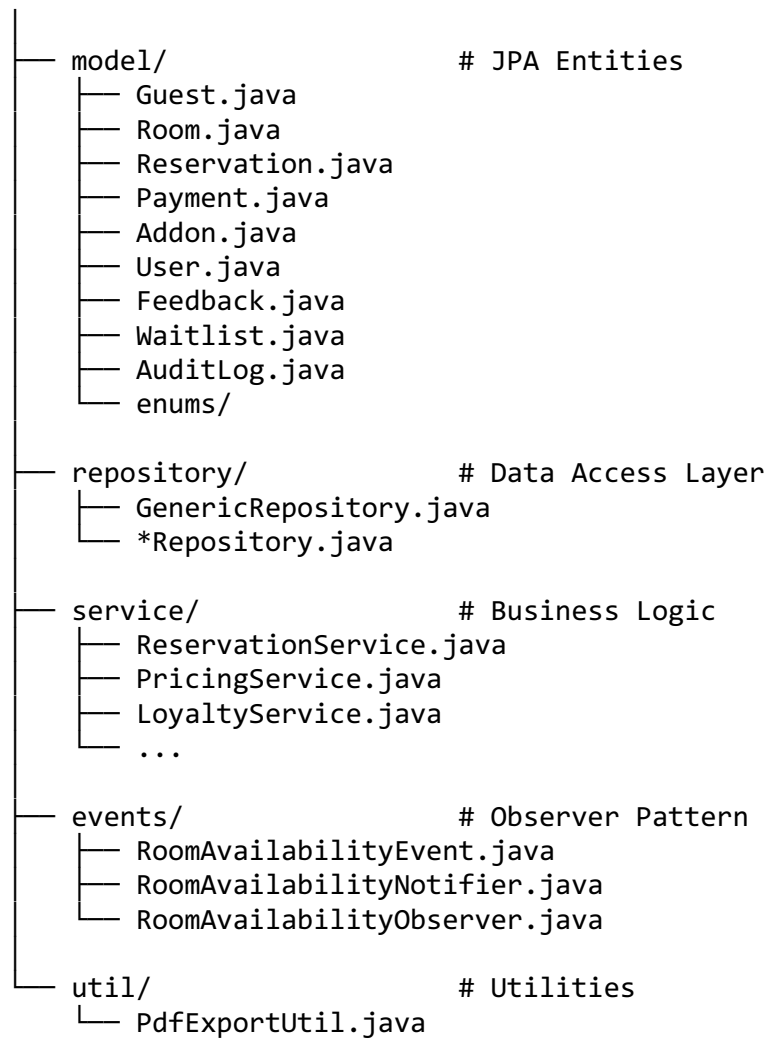
See: UML-deployment-diagram.png



3.4 Package Diagram

See: package-diagram.png

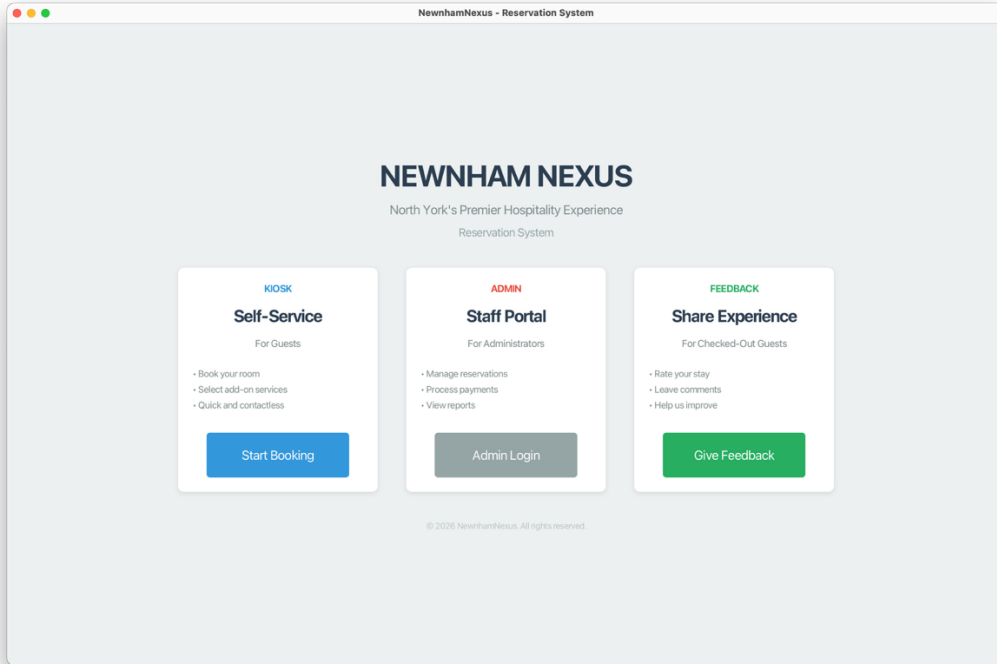




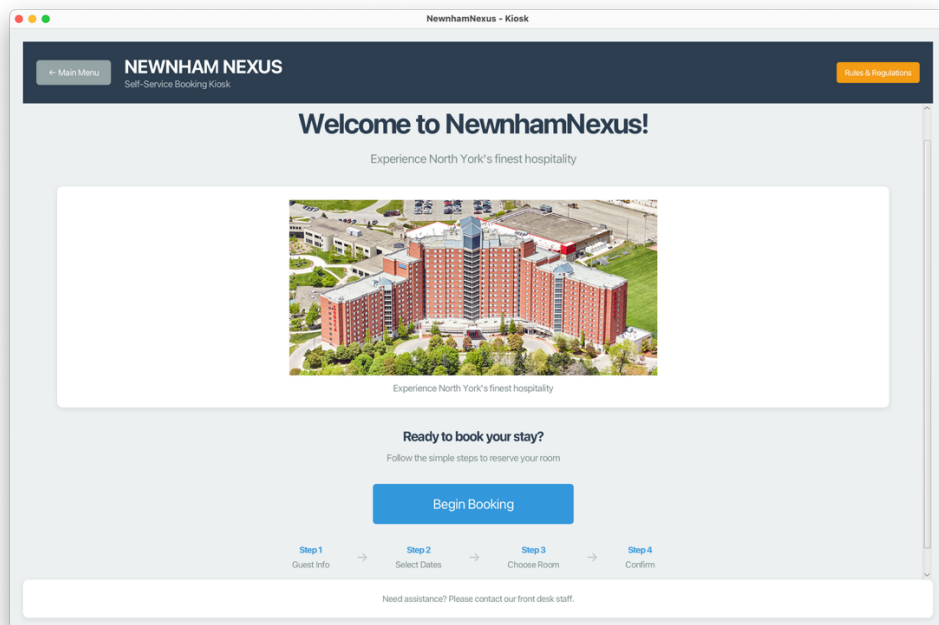
3.5 UI Screenshots

The application consists of three main interfaces:

1. **Launcher Screen** - Entry point with options for Kiosk, Admin, and Feedback



2. **Kiosk Screens** (6 steps) - Guest count, dates, room, add-ons, details, confirmation



NewnhamNexus - Guest Count

← Main Menu

NEWNHAM NEXUS

Step 1 of 6: Number of Guests

Rules & Regulations

1

2

3

4

5

6

Guests

Dates

Room

Add-ons

Details

Confirm

How many guests will be staying?

Adults

Ages 18 and above

-

2

+

Children

Ages 0-17

-

1

+

Total Guests

3

← Back

Continue →

NewnhamNexus - Select Dates

← Main Menu

NEWNHAM NEXUS

Step 2 of 6: Select Your Dates

Rules & Regulations

✓

2

3

4

5

6

Guests

Dates

Room

Add-ons

Details

Confirm

When would you like to stay?

Check-in Date

2026-04-06

Check-in time: 3:00 PM

→

Check-out Date

2026-04-11

Check-out time: 11:00 AM

Number of Nights

5

Guests

2 Adults, 1 Children

Note: Weekend rates (Fri-Sat) may differ from weekday rates.
Seasonal pricing may also apply during peak periods.

← Back

Continue →

NewnhamNexus - Select Room

← Main Menu

NEWNHAM NEXUS

Step 3 of 6: Select Your Room

Rules & Regulations

✓

✓

3

4

5

6

Guests

Dates

Room

Add-ons

Details

Confirm

Please check the room booking policies to ensure your selected room occupancy requirements.

Single Room

Up to 2 guests

* 1 Queen bed * 250 sq ft

Available: 5 rooms

\$100

per night

Quantity

0

+

Double Room

Up to 4 guests

* King or 2 Queens * 400 sq ft

Available: 5 rooms

\$150

per night

Quantity

0

+

Deluxe Room

Up to 4 guests

* King bed * Lake view * 500 sq ft

Available: 3 rooms

\$200

per night

Quantity

1

+

Penthouse Suite

Up to 6 guests

* King bed * Panoramic view * 800 sq ft

Available: 2 rooms

\$350

per night

Quantity

0

+

← Back

Continue →

NewnhamNexus - Add-Ons

← Main Menu

NEWNHAM NEXUS

Step 4 of 6: Add-On Services

Rules & Regulations

✓

✓

✓

4

5

6

Guests

Dates

Room

Add-ons

Details

Confirm

☒ **Daily Breakfast Buffet**

Full breakfast buffet with continental and hot options (per guest)

\$25

per night / per guest

☒ **Valet Parking**

Secure underground parking with valet service

\$30

per night

☐ **Spa Package**

60-minute massage and access to spa facilities (per guest)

\$75

per reservation / per guest

☒ **Airport Shuttle**

Round-trip shuttle service to/from North York area

\$50

per reservation

Price Impact Summary

Room Total (2 nights): \$1000.00

Add-ons Selected: \$250.00

Subtotal: \$1250.00

* Tax will be calculated at checkout

You can skip add-ons if you prefer. They can also be added later at the front desk.

← Back

Skip Add-ons

Continue →

NewnhamNexus - Guest Details

← Main Menu

NEWNHAM NEXUS

Step 5 of 6: Guest Information

Rules & Regulations

✓

✓

✓

✓

5

6

Confirm

Guests

Dates

Room

Add-ons

Details

Confirm

Enter Your Details

Please fill in the required information below

* indicates required field

First Name *

Dohyun

Last Name *

Lim

Email Address *

dohyunlim@test.com

Phone Number *

43712345678

Street Address

1750 Finch Ave E

City

North York

Country

Canada

Special Requests (optional)

Heater is needed.

← Back

Review Booking →

NewnhamNexus - Review Booking

← Main Menu

NEWNHAM NEXUS

Step 6 of 6: Review & Confirm

Rules & Regulations

✓

✓

✓

✓

✓

6

Confirm

Guests

Dates

Room

Add-ons

Details

Confirm

Review Your Booking

Guest Information

Name: Dohyun Lim

Email: dohyunlim@test.com

Phone: 43712345678

Guests: 2 Adults, 1 Children

Stay Details

Check-in: April 6, 2026 (3:00 PM)

Check-out: April 11, 2026 (11:00 AM)

Duration: 5 Nights

Room Selection

1 x Deluxe Room

Up to 4 guests, City view

(Multiple rooms)

Add-On Services

Price Estimate

Room Charges

Double Room × 2 nights

\$1000.00

Add-On Services

Wi-Fi

\$10.00

Breakfast

\$100.00

Subtotal

\$1250.00

Tax (13% HST)

\$162.50

ESTIMATED TOTAL

\$1412.50

* Final amount will be confirmed at checkout

Payment Information

Payment will be processed at the front desk during check-in or check-out. You may pay with cash, card, or loyalty points.

← Edit Booking

Confirm Reservation

3. Admin Dashboard - Navigation to all management screens

NewnhamNexus - Admin Login

NEWNHAM NEXUS HOTEL

Staff Portal

Administrator Login

Username

Password

☐ Remember me

Login

[Forgot password?](#)

Access Levels

Admin

Up to 10% discount

Manager

Up to 30% discount

← Back to Main Menu

[illegible]

4. Admin Screens - Reservations, Guests, Payments, Checkout, Waitlist, Feedback, Loyalty, Reports

NEWNHAM NEXUSHOTEL

adminADMIN

Dashboard

Reservations

Guests

Payments

Check-out

Waitlist

Feedback

Loyalty

Reports

Logout

Reservation Management

Create, modify, and manage reservations

+ New Reservation

Search by name, phone, or confirmation #

All

Date Range: From Date To Date

Find

Reset

Reservations

Showing 15 results

Conf. #	Guest Name	Phone	Room(s)	Check-in	Check-out	Total	Status
NN-3C5996DF	Test Testing	1234567890	1 room(s)	2026-03-26	2026-03-31	\$1440.75	Checked Out
NN-E754BC04	Test2 Testing2	1234567890	1 room(s)	2026-03-27	2026-04-02	\$1158.25	Checked Out
NN-7A60CDC0	Test3 Test3	1234567890	1 room(s)	2026-03-24	2026-03-26	\$192.10	Checked Out
NN-8TA3AD90	test4 test4	1234567890	1 room(s)	2026-03-24	2026-03-26	\$452.00	Cancelled
NN-8475449C	test6 test6	1234567890	1 room(s)	2026-03-24	2026-03-26	\$226.00	Cancelled
NN-58ADB952	test test	1234567890	1 room(s)	2026-03-25	2026-03-29	\$762.75	Cancelled
NN-82B5F3CB	Mike Lim	1234567890	1 room(s)	2026-04-09	2026-04-14	\$1497.25	Checked Out
NN-5D81F3CF	Testing1 Testing1	1234567890	1 room(s)	2026-04-05	2026-04-09	\$874.39	Checked In
NN-C82BE58B	Testing2 Testing2	1234567890	1 room(s)	2026-04-05	2026-04-07	\$384.20	Checked Out
NN-5A6FCC25	Testing3 Testing3	1234567890	1 room(s)	2026-04-05	2026-04-07	\$203.40	Checked In
NN-855F8046	Testing4 Testing4	1234567890	1 room(s)	2026-04-05	2026-04-14	\$3193.86	Checked Out
NN-74D7576A	Testing5 Testing5	012345679	1 room(s)	2026-04-06	2026-04-08	\$226.00	Checked Out
NN-16D5C24E	Testing6 Testing6	0123456789	1 room(s)	2026-04-06	2026-04-08	\$339.00	Checked Out
NN-A8F31230	Testing7 Testing7	1234567890	1 room(s)	2026-04-06	2026-04-08	\$226.00	Checked Out
NN-95FCBBA1	Dohyun Lim	43712345678	1 room(s)	2026-04-06	2026-04-11	\$1469.00	Confirmed

PreviousPage 1 of 5Next

Reservation Details

Select a reservation

Guest: -

Phone: -

Email: -

Room: -

Dates: -

Guests: -

Status: -

Billing Summary

Subtotal: \$0.00

Tax (13% HST): \$0.00

Total: \$0.00

Paid: \$0.00

Balance: \$0.00

Check-In Guest

Process Payment

Cancel Reservation

NEWNHAM NEXUSHOTEL

adminADMIN

Dashboard

Reservations

Guests

Payments

Check-out

Waitlist

Feedback

Loyalty

Reports

Logout

Guest Management

Search and manage guest information

Search by name, email, or phone

Loyalty Status

Search

Clear

Guests

Showing 17 guests

ID	Name	Email	Phone	Loyalty #	Points
1	Test Testing	test@testing.com	1234567890	-	0
2	Test2 Testing2	test2@testing2.com	1234567890	NN-61F428B0	11582
3	Test3 Test3	test3@test3.com	1234567890	NN-B6F7CB3B	1921
4	test4 test4	test4@test4.com	1234567890	-	0
5	test6 test6	test6@test6.com	1234567890	-	0
6	test test	test@test.com	1234567890	-	0
7	Mike Lim	test_mikelim@gmail.com	1234567890	-	0
8	Testing1 Testing1	testing1	1234567890	-	0
9	Testing2 Testing2	testing2@testing2.com	1234567890	-	0
10	Testing3 Testing3	Testing3@testing3.com	1234567890	-	0
11	Testing4 Testing4	Testing4@testing4.com	1234567890	-	0
12	Waitlist1	Waitlist1@Waitlist1.com	1234567890	-	0
13	Waitlist1	Waitlist1@Waitlist1.com	1234567890	-	0
14	Testing5 Testing5	Testing5@testing5.com	012345679	-	0
15	Testing6 Testing6	Testing6@testing6.com	0123456789	NN-5A71FC1A	3490
16	Testing7 Testing7	Testing7@testing7.com	1234567890	NN-406A91F3	2360
17	Dohyun Lim	dohyunlim@test.com	43712345678	NN-7E166294	100

Guest Profile

Select a guest

Email: -

Phone: -

Loyalty #: -

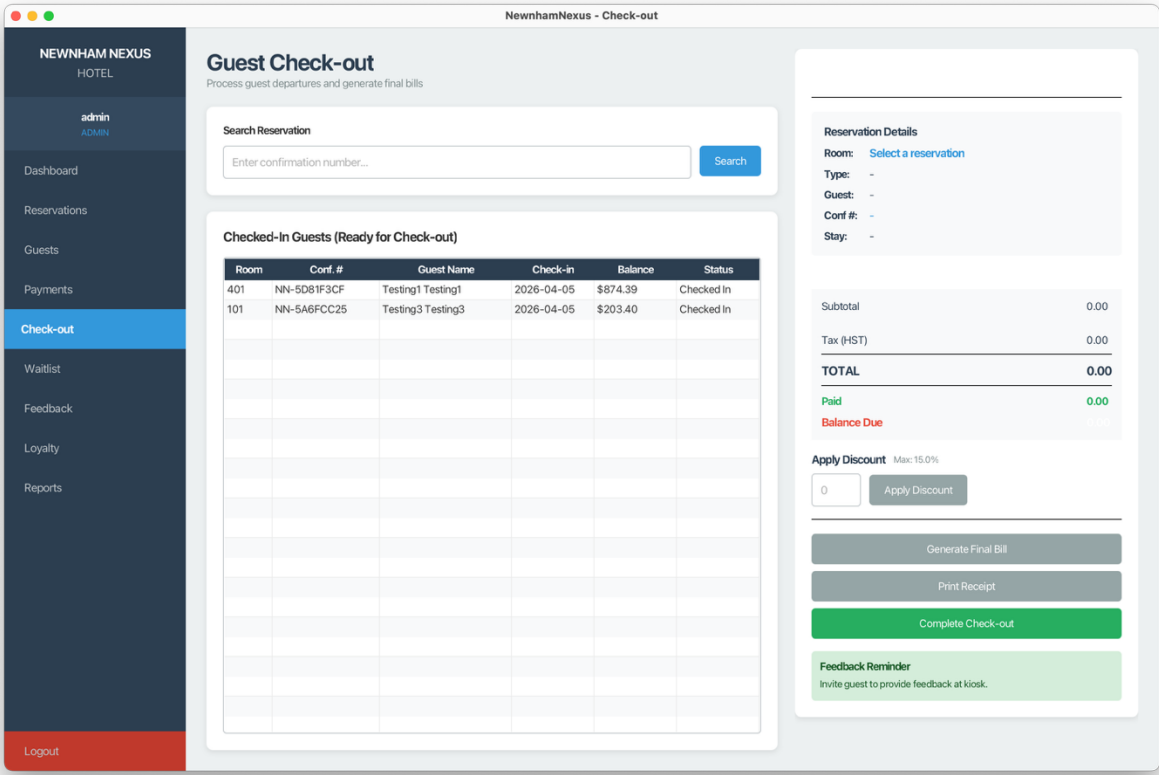
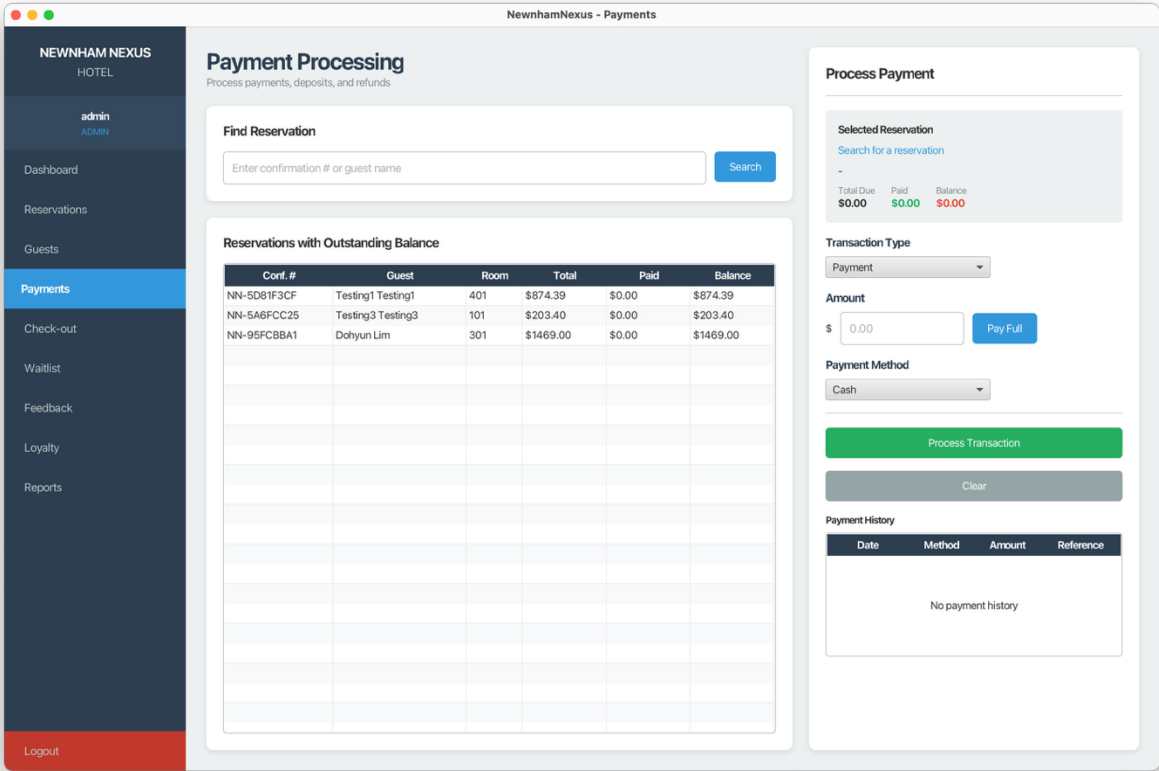
Points: 0

Reservation History

Select a guest to view reservations

View All Reservations

Enroll in Loyalty



5. Feedback Screens – Find Reservation, Details, Confirmation

The screenshot displays a web application window titled "NewnhamNexus - Feedback". The header features the "NEWNHAM NEXUS" logo and "Guest Feedback" text. The main heading is "We Value Your Feedback!" with the subtext "Please share your experience with us". Below this is a "Find Your Reservation" section with two radio buttons: "Confirmation Number" and "Phone Number". The "Phone Number" option is selected. A text input field contains the placeholder "e.g., 416-555-1234". A blue "Lookup" button is positioned to the right of the input field. A horizontal line separates the lookup section from a "← Back to Main Menu" button. At the bottom, a footer message reads "Thank you for staying with NewnhamNexus!".

NewnhamNexus - Feedback

NEWNHAM NEXUS
Guest Feedback

We Value Your Feedback!
Please share your experience with us

Find Your Reservation

☐ Confirmation Number ☒ Phone Number

[Lookup](#)

[← Back to Main Menu](#)

Thank you for staying with NewnhamNexus!

NewnhamNexus - Feedback

NEWNHAM NEXUS
Guest Feedback

We Value Your Feedback!
Please share your experience with us

Find Your Reservation

☐ Confirmation Number

☒ Phone Number

Lookup

Guest

Room

Stay

✓ Checked Out

Testing6 Testing6DOUBLE #201Apr 6, 2026 - Apr 8, 2026

How was your overall experience?

★

★

★

★

★

Very Good

What did you like most? (optional)

Cleanliness

Comfort

Service

Location

Value

Additional Comments (optional)

Thank you for great service.

Thank you for staying with NewnhamNexus!

NewnhamNexus - Thank You

NEWNHAM NEXUS
Feedback Received

✓

Thank You!
Your feedback has been submitted successfully.

Feedback Summary

★

★

★

★

★

Confirmation #: NNX-2026030501

Submitted: March 9, 2026

Your Opinion Matters!
We carefully review all feedback to improve our services.
We look forward to welcoming you back soon!

Submit Another Feedback

Return to Main Menu

Thank you for choosing NewnhamNexus!

4. Entity and Relationship Mapping

4.1 Entities (9 Tables + 2 Join Tables)

Entity	Table Name	Primary Key	Description
Guest	guests	id	Hotel guests/customers
Room	rooms	id	Hotel rooms
Reservation	reservations	id	Booking records
Payment	payments	id	Payment transactions
Addon	addons	id	Add-on services
User	users	id	Admin/Manager accounts
Feedback	feedbacks	id	Guest reviews
Waitlist	waitlist	id	Waitlist entries
AuditLog	audit_logs	id	Activity logs
-	reservation_rooms	-	Join table (Reservation-Room)
-	reservation_addons	-	Join table (Reservation-Addon)

4.2 Relationships

One-to-Many (1:N)

Parent	Child	Description
Guest	Reservation	One guest can have many reservations
Guest	Feedback	One guest can leave many feedbacks
Guest	Waitlist	One guest can have many waitlist entries
Reservation	Payment	One reservation can have many payments

Many-to-Many (M:N)

Entity 1	Entity 2	Join Table	Description
Reservation	Room	reservation_rooms	Reservation can have multiple rooms
Reservation	Addon	reservation_addons	Reservation can have multiple add-ons

4.3 Key JPA Annotations Used

// Entity and Table

@Entity

@Table(name = "guests")

public class Guest {

// Primary Key with Auto-Increment

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private Long id;

// Required Field

@Column(nullable = false)

private String firstName;

// Unique Constraint

@Column(unique = true)

private String loyaltyNumber;

// Enum Stored as String

@Enumerated(EnumType.STRING)

private RoomType type;

}

// Many-to-One Relationship

@Entity

public class Reservation {

@ManyToOne(fetch = FetchType.EAGER)

@JoinColumn(name = "guest_id")

private Guest guest;

}

// Many-to-Many with Join Table

@Entity

public class Reservation {

@ManyToMany(fetch = FetchType.EAGER)

@JoinTable(

name = "reservation_rooms",

joinColumns = @JoinColumn(name = "reservation_id"),

inverseJoinColumns = @JoinColumn(name = "room_id")

)

private List<Room> rooms;

}

4.4 Cascade and Fetch Strategy Notes

Relationship	Fetch Type	Cascade	Reason
Reservation → Guest	EAGER	None	Always need guest info with reservation
Reservation → Rooms	EAGER	None	Always need room info with reservation
Reservation → Addons	EAGER	None	Always need addon info with reservation
Reservation → Payments	LAZY	PERSIST	Load payments only when needed
Guest → Waitlist	EAGER	None	Quick access for notifications

5. Pattern Usage

5.1 Singleton Pattern

Purpose: Ensure single instance management for shared resources.

Implementations:

1. **EntityManagerFactoryProvider** - Single database connection factory

```
public class EntityManagerFactoryProvider {  
    private static EntityManagerFactory emf;  
  
    public static synchronized EntityManagerFactory getEntityManagerFactory()  
    {  
        if (emf == null) {  
            emf = Persistence.createEntityManagerFactory("HotelPU");  
        }  
        return emf;  
    }  
}
```


2. **LoyaltyService** - Single instance for loyalty point management
3. **AuditService** - Single instance for activity logging
4. **BookingSession** - Single instance for kiosk booking state

5.2 Factory Pattern

Purpose: Create room instances without exposing creation logic.

Implementation: RoomFactory and RoomPlanFactory

```
public class RoomFactory {
    public static Room createRoom(String roomNumber, RoomType type, int floor)
    {
        Room room = new Room();
        room.setRoomNumber(roomNumber);
        room.setType(type);
        room.setFloor(floor);
        room.setStatus(RoomStatus.AVAILABLE);
        room.setBasePrice(getBasePriceForType(type));
        room.setMaxOccupancy(getMaxOccupancyForType(type));
        return room;
    }

    private static double getBasePriceForType(RoomType type) {
        return switch (type) {
            case SINGLE -> 100.0;
            case DOUBLE -> 150.0;
            case DELUXE -> 200.0;
            case PENTHOUSE -> 350.0;
        };
    }
}
```

Usage: In DatabaseInitializer to create all initial room instances.

5.3 Strategy Pattern

Purpose: Dynamic pricing calculation based on day type.

Implementation:

```
// Strategy Interface
public interface PricingStrategy {
    double calculatePrice(double basePrice);
    String getStrategyName();
}
```

```

// Standard Pricing (Weekdays)
public class StandardPricingStrategy implements PricingStrategy {
    @Override
    public double calculatePrice(double basePrice) {
        return basePrice * 1.0; // No markup
    }
}

// Weekend Pricing (Friday/Saturday)
public class WeekendPricingStrategy implements PricingStrategy {
    @Override
    public double calculatePrice(double basePrice) {
        return basePrice * 1.25; // 25% markup
    }
}

// Context: PricingService
public class PricingService {
    public PricingStrategy getStrategyForDate(LocalDate date) {
        DayOfWeek day = date.getDayOfWeek();
        if (day == DayOfWeek.FRIDAY || day == DayOfWeek.SATURDAY) {
            return new WeekendPricingStrategy();
        }
        return new StandardPricingStrategy();
    }
}

```

5.4 Observer Pattern

Purpose: Notify admins when rooms become available after checkout.

Implementation:

```

// Event Data
public class RoomAvailabilityEvent {
    private final Room room;
    private final LocalDate availableFrom;
    private final LocalDate availableTo;
    // getters...
}

// Observer Interface
public interface RoomAvailabilityObserver {
    void onRoomAvailable(RoomAvailabilityEvent event);
}

// Notifier (Subject)

```

```

public class RoomAvailabilityNotifier {
    private final List<RoomAvailabilityObserver> observers = new ArrayList<>
();

    public void subscribe(RoomAvailabilityObserver observer) {
        observers.add(observer);
    }

    public void notifyRoomAvailable(RoomAvailabilityEvent event) {
        for (RoomAvailabilityObserver observer : observers) {
            observer.onRoomAvailable(event);
        }
    }
}

```

Usage: When checkout completes in AdminController, it publishes a RoomAvailabilityEvent. The waitlist observer checks for matching entries and notifies guests.

5.5 Decorator Pattern

Purpose: Dynamically add services to booking pricing.

Implementation:

```

// Base Interface
public interface BookingComponent {
    double getPrice();
    String getDescription();
}

// Concrete Component
public class BaseBooking implements BookingComponent {
    private final double basePrice;
    public double getPrice() { return basePrice; }
}

// Abstract Decorator
public abstract class BookingDecorator implements BookingComponent {
    protected BookingComponent wrapped;
    public BookingDecorator(BookingComponent wrapped) {
        this.wrapped = wrapped;
    }
}

// Concrete Decorator
public class ServiceDecorator extends BookingDecorator {
    private final Addon addon;
}

```

```

    private final int nights;

    @Override
    public double getPrice() {
        double addonPrice = addon.getPricingModel() == PricingModel.PER_NIGHT
            ? addon.getPrice() * nights
            : addon.getPrice();
        return wrapped.getPrice() + addonPrice;
    }
}

```

Usage: In `PricingService.calculateTotalWithAddons()` to build up the final price with all selected add-ons.

6. Business Rules

6.1 Room Occupancy Rules

Room Type	Max Occupancy	Notes
Single	2 guests	Adults + Children combined
Double	4 guests	Adults + Children combined
Deluxe	4 guests	Adults + Children combined
Penthouse	6 guests	Adults + Children combined

6.2 Pricing Rules

Rule	Description
Base Pricing	Single: \$100, Double: \$150, Deluxe: \$200, Penthouse: \$350
Weekend Multiplier	Friday/Saturday nights: 1.25x base price
Tax Rate	13% HST applied to subtotal
Add-on Pricing	PER_NIGHT or PER_RESERVATION based on addon type

6.3 Discount Rules

Role	Maximum Discount	Notes
ADMIN	15%	Applied at checkout
MANAGER	30%	Applied at checkout

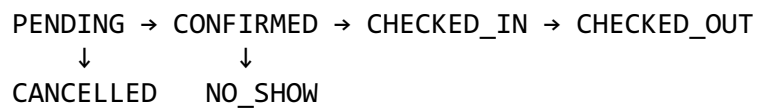
6.4 Loyalty Program Rules (Nexus Rewards)

Rule	Value
Points Earned	10 points per \$1 spent
Points Redemption	100 points = \$1 discount
Welcome Bonus	100 points on enrollment
Minimum Redemption	100 points

6.5 Feedback Rules

Rule	Description
Rating Range	1-5 stars
One Per Reservation	Only one feedback per reservation allowed
Requires Confirmation #	Guest must provide valid confirmation number
Sentiment Tags	Excellent, Good, Average, Poor, Terrible (auto-assigned)

6.6 Reservation Status Flow



7. Security and Logging

7.1 Authentication

Password Hashing: BCrypt with default work factor

```
// Hashing (during user creation)
String hash = BCrypt.hashpw(plainPassword, BCrypt.gensalt());

// Verification (during login)
boolean valid = BCrypt.checkpw(plainPassword, storedHash);
```

7.2 Role-Based Access Control

Feature	ADMIN	MANAGER
View Reservations	✓	✓

Feature	ADMIN	MANAGER
Create Reservations	✓	✓
Process Payments	✓	✓
Apply Discount (15%)	✓	✓
Apply Discount (30%)	✗	✓
View Reports	✓	✓
Export Reports	✓	✓

7.3 Logging Configuration

File Logging: - Location: ./logs/hotel.log - Rotation: 1MB per file, 10 files maximum - Format: [TIMESTAMP] [LEVEL] [CLASS] - MESSAGE

Database Logging (AuditLog): - Actor: Username who performed action - Action: CREATE, UPDATE, DELETE, LOGIN, LOGOUT, PAYMENT - Entity Type: What was affected - Message: Human-readable description

7.4 Exception Handling

All exceptions are caught and displayed to users via JavaFX Alert dialogs:

```
try {
    // Operation
} catch (Exception e) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle("Error");
    alert.setHeaderText("Operation Failed");
    alert.setContentText(e.getMessage());
    alert.showAndWait();

    // Log the error
    logger.error("Operation failed: " + e.getMessage(), e);
}
```

8. Export and Reporting

8.1 Report Types

Report	Description	Filters
Revenue Report	Total revenue by period	Date range (day/week/month)
Occupancy Report	Room utilization statistics	Date range, room type
Feedback Summary	Rating averages and comments	Date range, rating, sentiment
Activity Logs	System activity audit trail	Date range, actor, action

8.2 Export Formats

Format	Implementation	Use Case
CSV	Built-in Java file writing	Data analysis in Excel
PDF	Apache PDFBox library	Printable reports
TXT	Built-in Java file writing	Activity logs

8.3 Sample Report Structure

Revenue Report (CSV):

Period,Room Revenue,Addon Revenue,Total Revenue,Reservations
2026-03-01,1500.00,250.00,1750.00,5
2026-03-02,2100.00,180.00,2280.00,7

Occupancy Report (PDF):

NEWNHAM NEXUS - Occupancy Report
Period: March 1-31, 2026

Room Type	Available	Occupied	Occupancy Rate
-----	-----	-----	-----
Single	155	120	77.4%
Double	155	145	93.5%
Deluxe	93	85	91.4%
Penthouse	62	58	93.5%

8.4 PDF Export Implementation

```
public class PdfExportUtil {
    public static void exportDocument(String title, List<String> content, String filePath)
        throws IOException {
        try (PDDocument document = new PDDocument()) {
            PDPage page = new PDPage();
            document.addPage(page);

            try (PDPageContentStream stream = new PDPageContentStream(document, page)) {
                stream.setFont(PDType1Font.HELVETICA_BOLD, 18);
                stream.beginText();
                stream.newLineAtOffset(50, 750);
                stream.showText(title);
                stream.endText();

                // Write content lines...
            }

            document.save(filePath);
        }
    }
}
```

9. Challenges and Learnings

9.1 Team Challenges

Challenge 1: Code Merging and Version Control

Problem: Multiple team members working on different features simultaneously resulted in numerous merge conflicts (20+ conflicting files at one point).

Solution: Established a “primary version” approach where one member’s UI/UX was the base, and architectural components were carefully merged. Improved communication about file ownership and used feature branches consistently.

Challenge 2: Integrating JavaFX Frontend with Hibernate/JPA Backend

Problem: Entity objects weren't persisting correctly, lazy loading caused exceptions, and EntityManager lifecycle management was challenging.

Solution: Created service classes as intermediaries between controllers and repositories. Switched to EAGER fetch for critical relationships and ensured `repository.update()` was called when modifying entities.

Challenge 3: Implementing Design Patterns Correctly

Problem: Initially had pattern classes implemented but they weren't actually being used in the code.

Solution: Went through each pattern systematically: - Factory: Used in DatabaseInitializer - Strategy: Wired into PricingService - Observer: Connected to checkout flow - Decorator: Integrated into addon pricing

Challenge 4: UI/UX Consistency

Problem: Different screens had inconsistent styling, sidebar layouts, and color schemes.

Solution: Created shared `styles.css` and established conventions for common components. Did a final pass to ensure all admin screens had consistent navigation and styling.

Challenge 5: Database Schema Evolution

Problem: Adding new fields to entities caused issues with existing H2 database data.

Solution: Used Hibernate's "update" schema generation and handled null values carefully with wrapper types and null-safe getters.

9.2 Individual Reflections

See: `reflect.txt` for detailed individual reflections from each team member.

Summary of Key Learnings:

1. **Design Patterns** - Implementing patterns isn't just about creating classes; they must be meaningfully integrated into the codebase.
2. **JPA/Hibernate** - Understanding entity lifecycle (transient, managed, detached) is crucial for proper persistence.
3. **Git Workflow** - Merge conflicts can be minimized with good communication and branch management.

4. **Separation of Concerns** - Service classes make code more testable and maintainable.
5. **Security** - BCrypt for password hashing, proper salting, and work factors are essential.
6. **Database Design** - Early decisions have long-lasting effects; changing relationships later requires updating multiple files.
7. **Team Collaboration** - Writing clean, readable code with comments and naming conventions helps when others need to work with the code.

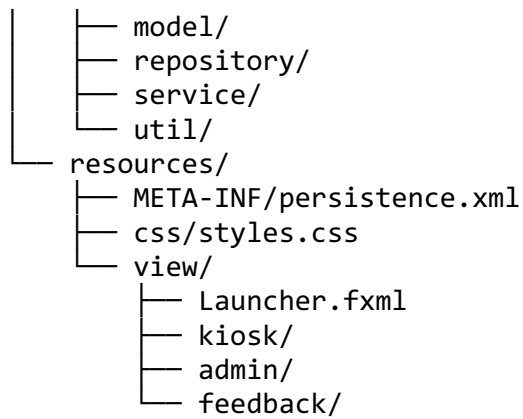
9.3 Suggestions for Future Improvements

1. **Multi-tenant Support** - Allow multiple hotel properties
2. **Email Notifications** - Send actual confirmation emails
3. **Mobile App** - Develop companion mobile application
4. **Online Payments** - Integrate Stripe/PayPal
5. **Room Images** - Add photo gallery for rooms
6. **Multi-language** - Support for multiple languages
7. **API Layer** - RESTful API for third-party integrations

Appendices

A. File Structure

```
/Project
├── pom.xml
├── README.md
├── DATABASE_DESIGN.md
├── ProjectDocumentation.md
├── reflect.txt
├── UML-deployment-diagram.png
├── package-diagram.png
├── data/
│   └── hoteldb.mv.db
├── logs/
│   └── hotel.log
├── src/main/
│   ├── java/com/hotel/
│   │   ├── app/
│   │   ├── controller/
│   │   └── events/
```



B. How to Run

1. **Prerequisites:** Java 17+, Maven 3.6+
2. **Build:** `mvn clean install`
3. **Run:** `mvn javafx:run`
4. **Default Accounts:**
 - Admin: admin / admin123
 - Manager: manager / manager123

C. References

- JavaFX Documentation: <https://openjfx.io/>
- Hibernate ORM: <https://hibernate.org/orm/>
- H2 Database: <https://h2database.com/>
- Apache PDFBox: <https://pdfbox.apache.org/>
- BCrypt: <https://github.com/jeremyh/jBCrypt>